

# APPENDICE – Spring WebFlux

## Introduzione a Spring WebFlux

**Il punto di partenza: come funziona un server tradizionale.** Nelle applicazioni **MVC tradizionali** viene creato **un nuovo servlet thread per ogni request** che arriva. Quel thread poi fa partire tutta una serie di altri processi/thread che si occupano delle operazioni di **input/output** — per esempio verso il database — e **per tutto quel tempo il thread del server resta bloccato in stato di attesa**, quindi non può fare nient'altro.

Questa è l'**elaborazione sincrona** della richiesta. E qui c'è la conseguenza importante:

Il server ha un numero **limitato e finito** di server thread, e quel numero rappresenta la **capacità massima del server di elaborare richieste**.

Questa è la struttura di un server normale in Spring MVC: Tomcat, WildFly. Funzionamento sincrono, un server thread per ogni request.

**L'alternativa: elaborazione asincrona.** Nel modello asincrono **non c'è nessun thread in attesa**. C'è **un unico thread che riceve le richieste** (il *request thread*), che **delega** le varie richieste a un **pool di thread**. Sono quelli del pool a occuparsi della gestione delle singole request, e quando la richiesta è soddisfatta viene eseguita la **callback**: viene cioè richiamato chi la richiesta l'aveva fatta.

Il confronto che fa il docente:

|                                                                                                                                                                                                                                                                                                                                                                                                                      | Sincrono                                       | Asincrono                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|------------------------------------------|
| Il chiamante                                                                                                                                                                                                                                                                                                                                                                                                         | resta <b>bloccato</b> in attesa della response | <b>si stacca e continua a fare altro</b> |
| Il thread server                                                                                                                                                                                                                                                                                                                                                                                                     | occupato per tutta la durata                   | libero, il lavoro va a un pool           |
| Come arrivano i dati                                                                                                                                                                                                                                                                                                                                                                                                 | come valore di ritorno                         | tramite <b>notifica / callback</b>       |
| <b>Cosa vuol dire "reattivo".</b> Il termine <i>reattivo</i> si riferisce a modelli di programmazione <b>costruiti attorno alla reazione ai cambiamenti</b> . Nello stile reattivo: facciamo una richiesta di risorse, <b>iniziamo a fare altre cose</b> — quindi le chiamate sono <b>non bloccanti</b> — e <b>quando i dati torneranno disponibili riceveremo una notifica</b> insieme alle informazioni richieste. |                                                |                                          |

È un modello particolarmente utile per le applicazioni che lavorano con un **flusso di dati in stream**.

**Le Reactive Streams API.** In Java esiste per questo una specifica chiamata **Reactive Streams API**, disponibile **da Java 9 in poi**, di cui esistono varie implementazioni: **RxJava** e **Project Reactor**. La specifica si compone di **quattro interfacce**:

| Interfaccia         | Ruolo                                                                                                                                                                  |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Publisher</b>    | l'editore: <b>emette una sequenza di eventi</b> verso i subscriber, in base alle richieste ricevute da questi. Ci si abbona tramite il metodo <code>subscribe()</code> |
| <b>Subscriber</b>   | l'abbonato: <b>riceve ed elabora</b> gli elementi emessi dal publisher. Ha <b>quattro metodi</b> per gestire i vari tipi di risposta                                   |
| <b>Subscription</b> | definisce <b>la relazione tra un publisher e un subscriber</b> : serve per indicare che vogliamo i dati, o per <b>annullare l'abbonamento</b>                          |
| <b>Processor</b>    | può fungere <b>sia da publisher che da subscriber</b>                                                                                                                  |

Un editore può avere **più abbonati**. (Nel riepilogo il docente dice per sbaglio che il processor "può fungere sia da abbonato che da subscriber": intendeva *publisher e subscriber*, come detto correttamente poco prima.)

**Che cos'è Spring WebFlux.** È una **versione parallela di Spring MVC** che supporta **stream reattivi non bloccanti**. Internamente:

- usa la libreria **Project Reactor**;
- usa di default **Netty** come server reattivo non bloccante.

**Mono e Flux.** WebFlux usa due tipi di publisher, ed è la distinzione da tenere a mente:

| Publisher   | Quanti elementi                    |
|-------------|------------------------------------|
| <b>Mono</b> | da <b>0 a 1</b> elemento           |
| <b>Flux</b> | da <b>0 a N</b> elementi asincroni |

Il **Flux** **può essere infinito**: può continuare a emettere elementi per sempre. Può anche restituire una sequenza finita di elementi, e in quel caso arriva una **notifica di completamento** quando ha restituito tutti i suoi elementi.

Il pattern applicativo è questo: **si costruiscono controller che restituiscono un Mono o un Flux**, e poi si lavora su quell'oggetto per essere avvisati quando i dati asincroni sono disponibili.

---

## Spring WebFlux

**Creazione del progetto.** *New* → *Spring Starter Project*, e nelle dipendenze si aggiungono:

- **Spring Reactive Web** — il modulo WebFlux;
- **Spring Web** — perché servono i controller.

**Il repository.** Si crea una classe `EmployeeRepository`. Prima però serve la classe `Employee`, che è banalmente **una classe composta da id e name**. Nel repository c'è una **HashMap che funge da sorgente di dati**, con dentro una decina di impiegati.

I metodi sono due (il docente ne aveva scritto un terzo di `update`, ma lo elimina in diretta "*così lo facciamo ancora più semplice*" — è coerente con il suo modo di lavorare: "*io al solito cerco di fare degli esempi il più semplice possibile*");

- **ricerca per id** — prende dalla map l'impiegato con quell'id e **lo inserisce dentro un `Mono<Employee>`**;
- **findAll** — restituisce **dentro un `Flux<Employee>`** tutti gli impiegati contenuti nella map.

Ricostruito, il repository è questo:

```
@Repository
```

```
public class EmployeeRepository {
```

```
    private static final Map<String, Employee> EMPLOYEES = new HashMap<>();
```

```

static {
    EMPLOYEES.put("1", new Employee("1", "Mario"));
    EMPLOYEES.put("2", new Employee("2", "Luigi"));
    // ... una decina di impiegati
}

public Mono<Employee> findEmployeeById(String id) {
    return Mono.just(EMPLOYEES.get(id));
}

public Flux<Employee> findAllEmployees() {
    return Flux.fromIterable(EMPLOYEES.values());
}
}

```

**Il controller REST lato server.** È un `@RestController` che **prende l'id dal path** e richiama semplicemente il repository, restituendo l'oggetto `MONO`. Analogamente il metodo su tutti gli impiegati restituisce il `FLUX`.

`@RestController`

```

public class EmployeeController {

    @Autowired
    private EmployeeRepository repository;

    @GetMapping("/employees/{id}")
    public Mono<Employee> getEmployeeById(@PathVariable String id) {
        return repository.findEmployeeById(id);
    }

    @GetMapping("/employees")
    public Flux<Employee> getAllEmployees() {
        return repository.findAllEmployees();
    }
}

```

```
}  
}
```

La catena da tenere a mente: **repository che fornisce oggetti Mono/Flux** → **servizio REST che richiama il repository e a sua volta restituisce quegli oggetti.**

**Il lato client: WebClient.** Poi c'è un altro servizio REST che funge da client. Nel metodo `helloMono` si crea un oggetto **WebClient** e si richiama l'URL appena visto **passandogli come parametro id il valore 1**. Questo restituisce un Mono che al suo interno avrà l'Employee con id 1.

A quel punto **ci si abbona** con `subscribe()`, dando **due comportamenti**:

- cosa fare **in caso di successo** — qui semplicemente stampare la risposta;
- cosa fare **in caso di errore** — stampare l'errore.

@RestController

public class WebClientController {

@GetMapping("/hellomono")

public String helloMono() {

WebClient client = WebClient.create("http://localhost:8080");

Mono<Employee> employeeMono = client.get()

.uri("/employees/{id}", 1)

.retrieve()

.bodyToMono(Employee.class);

employeeMono.subscribe(

employee -> System.out.println("Risposta: " + employee),

error -> System.out.println("Errore: " + error)

);

System.out.println("invocazione fatta");

return "invocazione fatta";

}

@GetMapping("/helloflux")

```

public String helloFlux() {
    WebClient client = WebClient.create("http://localhost:8080");

    Flux<Employee> employeeFlux = client.get()
        .uri("/employees")
        .retrieve()
        .bodyToFlux(Employee.class);

    employeeFlux.subscribe(
        employee -> System.out.println("Risposta: " + employee),
        error -> System.out.println("Errore: " + error)
    );

    System.out.println("invocazione fatta");
    return "invocazione fatta";
}
}

```

Nota che in entrambi i casi bisogna dire esplicitamente **di che tipo sono gli oggetti restituiti** (`Employee.class`) — il docente ci torna sopra due volte.

**La demo con Postman, e cosa dimostra.** Si avvia il progetto Spring Boot e si chiama prima `/hello`: in console la **callback viene chiamata una sola volta**, perché il `MONO` può avere **da 0 a 1 elementi**. Poi basta **cambiare l'URL** e chiamare la versione `FLUX`: qui la callback viene **richiamata più volte**, man mano che i vari impiegati vengono generati.

Il particolare su cui il docente insiste — ed è il vero punto della lezione:

Guardando il codice si vede che dopo `subscribe()` **il metodo non è rimasto bloccato in attesa**: è andato avanti, ha stampato *"invocazione fatta"* ed è terminato. **Solo dopo**, man mano che arrivavano i dati, veniva richiamato il metodo di callback: oggetto uno, oggetto due, oggetto tre.

Questo è il comportamento asincrono osservato: **non siamo bloccati sulla chiamata**.

**L'analogia finale.** Quella con cui chiude la lezione, ed è la più efficace per fissare il concetto:

- **chiamata sincrona = il telefono** — per parlare con qualcuno dovete essere presenti **nello stesso istante**;
- **chiamata asincrona = l'email** — mando il messaggio, e **lui quando la legge con calma mi risponde**.